

ADR-001: Host for Contentstack Demonstrations

Status: Proposed
Date: September 30, 2025
Team 3 Project Stallion
Jake Terry

Context

The portal will be built using Contentstack for content management, Next.js for the front end, and Supabase for authentication and backend services.

A key decision is where and how to host the Next.js application. Hosting must deliver content globally with high performance, and remain cost-effective as usage scales, and integrate seamlessly with the chosen techstack.

Key Non-Functional Requirements (NFRs)

- Performance: Ensure fast load times and smooth rendering of content. API responses should generally return within < 300 ms under normal load.
- Scalability: Support growth in number of users (internal + partners) and content volume without major re-architecture.
- Availability: Aim for high uptime ($\geq 99.5\%$). Ensure redundancy in backend services (Supabase, Contentstack) and graceful degradation if an external service is temporarily unavailable.
- Reliability: Support streaming and download of demo videos without service degradation.

Options

1. Vercel (Chosen)

Pros: Offers native support for Next.js and simple deployment.

Cons: Can be expensive at scale, less flexibility for custom infrastructure.

2. AWS

Pros: Enterprise-grade scalability Easy to integrate with enterprise systems.

Cons: Requires high operational overhead (cloud expertise to configure and manage). Lacks seamless Next.js support.

3. Netlify

Pros: Developer friendly with simple deployments.

Cons: Fewer advanced Next.js optimizations compared to Vercel.

Decision

We will pursue Option 1: Host portal with Vercel. Vercel is the native platform for Next.js, offering seamless support for server-side rendering (SSR), incremental static regeneration (ISR), and middleware without custom configuration. Integration with GitHub and GitLab enables automatic builds and preview environments for every pull request, accelerating iteration for Sales Engineering and Demo Engineering teams. The serverless model scales automatically as more employees and partners use the portal, without requiring manual infrastructure adjustments.

Consequences

Positive:

- Teams can deploy and iterate rapidly with Vercel's native Next.js integration and automated CI/CD. Demo materials and features reach end-users (Sales, Partners) more quickly.
- Global edge network reduces latency, ensuring smooth delivery of demo instructions, UI assets, and content.
- Encrypted connections (HTTPS/TLS), secret management, and integration with Supabase authentication provide strong baseline security

Negative:

- Deep integration with Next.js features ties deployments closely to Vercel's ecosystem.
- Dependency on Supabase and Contentstack uptime.
- Usage-based pricing can increase quickly if demo videos or resource-heavy assets are hosted or proxied directly through Vercel.

Risks and Mitigations

- Risk: Vercel's usage-based pricing can increase significantly if large demo videos or heavy traffic are served directly through the platform.

Mitigation: Store and deliver videos through Contentstack or a dedicated CDN (e.g., Cloudflare R2, AWS S3 + CloudFront). Monitor usage and set budget alerts.

- Authentication failures (Risk): Supabase login may fail or break workflows.

Mitigation: Rigorous testing, fallback SSO mechanisms, monitoring and alerts.

- Low adoption (Risk): Users may continue using legacy sources.

Mitigation: Provide training, intuitive UX, and gradually deprecate scattered storage.

- Risk: The portal depends on multiple external services (Supabase, Contentstack, Vercel). Outages in any service could disrupt access for employees and partners.

Mitigation: Build in graceful degradation (e.g., cached demo instructions, fallback UI). Monitor service health and configure alerting to quickly respond to downtime

References

- Contentstack CMS: <https://www.contentstack.com>
- Supabase authentication docs: <https://supabase.com/docs>
- Next.js documentation: <https://nextjs.org/>